

Evaluación Globalizadora B

Sistemas Operativos de Propósito General

López Franco Alejandro, Matias Gabriel Pietras
Universidad Tecnológica Nacional Facultad Regional Avellaneda
francoalelopez@gmail.com
Mati_pietras@yahoo.com.ar

A. Índice	pág. 1
B. Abstract	pág. 1
C. Fundamentación Teórica	pág. 1
C.1. Fundamentación teórica del hardware	pág. 1
C.2. Fundamentación teórica del software	pág. 1
C.3. Procesamiento de señales	pág. 1
D. Objetivos	pág. 2
E. Alcance	pág. 2
F. Hardware	pág. 2
F.1. Módulos	pág. 2
G. Software	pág. 4
G.1 Tareas	pág. 4
G.2 Sistema operativo de Propósito general	pág. 5
G.3 Interfaz de usuario	pág. 5
G.4 Control PID	pág. 5
H. Conclusiones	pág. 5
J. Referencias	pág. 6
Anexos	pág. 7
Anexo A: Guía de código	pág. 7
Anexo B: Interacción entre EGA y EGB	pág. 7
Anexo C: Diagrama en bloques	pág. 8
Anexo D: PCB	pág. 9
Anexo E: Diagrama esquemático	pág. 9
Anexo F: Pinout Raspberry pi pico 2W	pág. 10
Anexo G: Pinout Raspberry pi 5	pág. 10
Anexo H: Test points	pág. 11
Anexo I: Listado de comandos implementados	pág. 12
Anexo J: Guía de usuario	pág. 13

B. Abstract— En este trabajo se desarrolló un dispositivo capaz de controlar la iluminación ambiental con una iluminancia constante o con variación lineal. El proyecto incluye el desarrollo de su driver para la interacción con una aplicación de usuario. La primera fase (EGA) consistió en un controlador de iluminación implementado con un microcontrolador RP2350 (M33), programado en C en el IDE Visual Studio Code (v1.103.2) utilizando la extensión oficial de Raspberry Pi Pico (v0.18.0) con el SDK C/C++ (v2.2.0), y operando con el sistema operativo en tiempo real FreeRTOS. La segunda fase (EGB), objeto de este informe, se centra en establecer una comunicación UART a nivel de kernel con el procesador BCM2712 (ARM Cortex-A72, ARMv8, 64 bits) de una Raspberry Pi 5, que ejecuta Raspberry Pi OS de 64 bits basado en Linux kernel v6.12.

Palabras clave: FreeRTOS, GPOS, Raspberry Pi 4 B, Pi Pico 2, control, comunicación, UART, filtros estadísticos.

C. FUNDAMENTACIÓN TEÓRICA

C.1 Fundamentación teórica del hardware

En este trabajo se ha seleccionado el fototransistor TEMENT6000 como sensor principal de luz, debido a su naturaleza completamente analógica, que le confiere una alta velocidad de respuesta en el orden de los microsegundos. Además, posee un comportamiento lineal en el rango de 100 a 1000 lux, lo que facilita su integración en sistemas de control.

Como complemento, se incorpora el sensor digital BH1750, que opera mediante el protocolo de comunicación I²C y ofrece mediciones de iluminancia con mayor precisión, aunque con una menor velocidad de respuesta (120 ms). Debido a esta diferencia, se emplea el BH1750 como referencia para la calibración de las lecturas del TEMENT6000, logrando así un sistema que combina la rapidez de un sensor analógico con la exactitud de uno digital.

C.2 Fundamentación teórica del software

Para el desarrollo del sistema de control propuesto, se ha optado por implementar el sistema operativo en tiempo real FreeRTOS, encargado de gestionar las tareas del control de iluminación. Paralelamente, se utiliza un sistema operativo de propósito general basado en el kernel de Linux para la interfaz de usuario, permitiendo una separación funcional entre el procesamiento en tiempo real y la interacción con el usuario. FreeRTOS facilita la ejecución concurrente de múltiples tareas mediante su planificador (scheduler), que organiza la ejecución según prioridades definidas.

En el entorno Linux, se desarrollará un módulo de kernel encargado de controlar el puerto UART, permitiendo la comunicación con el sistema embebido que gestiona el control de iluminación. Para la adquisición de datos, se emplea un convertidor analógico-digital (ADC) configurado mediante interrupciones, lo que garantiza una tasa de muestreo constante de 800 Hz, necesaria para un control rápido y estable.

C.3 Procesamiento de señales

Las mediciones del TEMENT6000 se deberán ajustar a través de una constante de calibración, la cual se obtendrá comparando sus mediciones con las del BH1750 en su tramo lineal y promediando sus valores para mitigar el ruido lo más posible. Para ello se ingresó a lazo abierto una rampa y se obtuvo el parámetro de calibración.

$$calib_{param} = \frac{prom_{bh1750}}{prom_{temt6000}}$$

$$lux = calib_{param} * Voltage$$

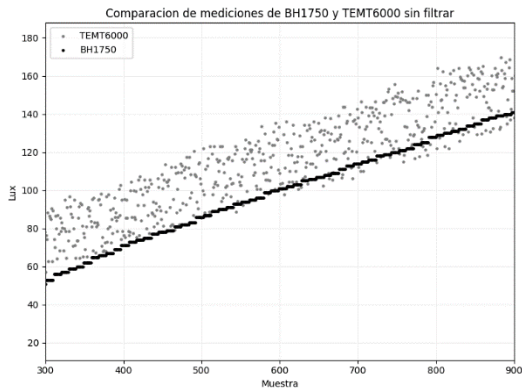


Fig 1. Comparación de mediciones entre BH1750 y TMT6000 sin filtrar muestreado a 800hz

Las mediciones obtenidas mediante el fototransistor TMT6000 presentan un nivel considerable de ruido (ver Fig 1), lo cual representa una problemática relevante, especialmente al encontrarse dicho ruido en el lazo de realimentación del sistema. Se optó por implementar un filtro de Kalman con fines académicos y de análisis, utilizando un modelo de predicción basado en un proceso de tipo random walk. Para mejorar la calidad de dicha estimación, se incorporan las mediciones del sensor BH1750, que proporciona datos cada 120 ms, correspondiente al periodo de muestreo configurado. Estas muestras se emplean para calcular dinámicamente la ganancia del filtro de Kalman en cada ciclo.

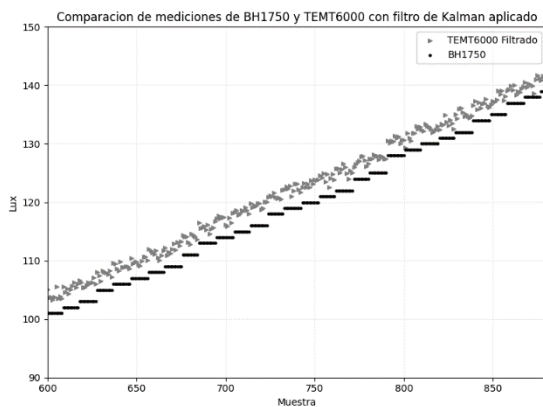


Fig 2. Comparación de mediciones entre BH1750 y TMT6000 con filtro de Kalman aplicado muestreado a 800hz.

D. OBJETIVOS

- El desarrollo de un driver para un periférico dentro de un GPOS.
- El uso de device trees en un GPOS.
- La comunicación de dos sistemas operativos con principios de funcionamiento diferentes.
- Coordinar programas que corran a nivel de kernel y usuario en un GPOS.

- Medir la luminosidad ambiental.
- Control de iluminación constante en entorno controlado.

E. ALCANCE

Se pretende medir la iluminación a una precisión del +/- 20%, compatible con el Decreto 351/79, dentro del rango de 100 a 1000 lux. El prototipo se hará dentro de una caja la cual tendrá una abertura modificable para validar el comportamiento del controlador ante perturbaciones.

F. HARDWARE

F.1. Módulos

Se detallarán los módulos utilizados en el proyecto y sus características más relevantes para el proyecto.

- Placa de desarrollo Pi Pico 2 W
 - Microcontrolador rp2350 (Cortex M33).
 - Tensión de alimentación 3,3V.
 - ADC de 3 canales de 12 bits.
 - Frecuencia de funcionamiento de hasta 120 Mhz.
 - Memoria Flash 4 Mbytes.
 Para observar el pinout ver Anexo E.

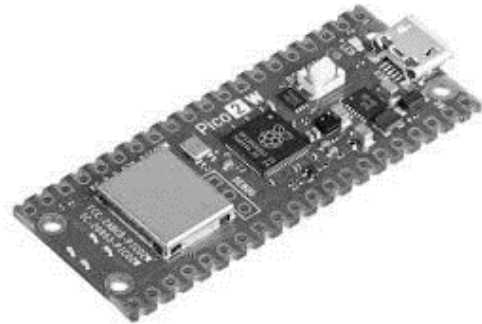


Fig 3. Placa de desarrollo Raspberry Pi Pico 2

- Fototransistor TMT 6000

Este fototransistor será responsable de las mediciones de variaciones rápidas de lux por su rápida respuesta, este módulo cuenta con una resistencia la cual nos sirve para medir su tensión desde el ADC y lograr una medición de luminosidad ambiente.

- Rango entre 100 a 1500 lux.
- Rango espectral entre 390-700 nm, con pico alrededor de 570 nm, similar al ojo humano.
- Ángulo de detección +/- 60°.

Pinout:

- **V:** Pin de alimentación (3V3).
- **G:** Pin de tierra.
- **S:** Pin de señal.

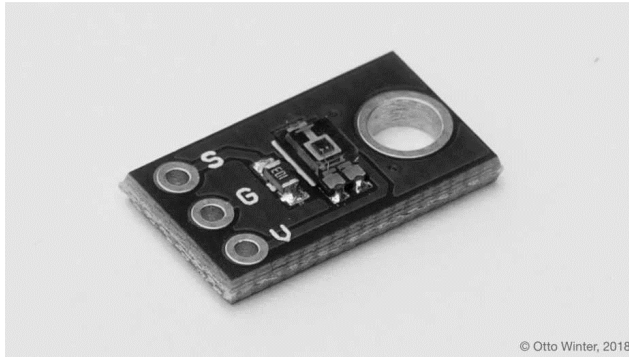


Fig 4. Sensor TEMT6000

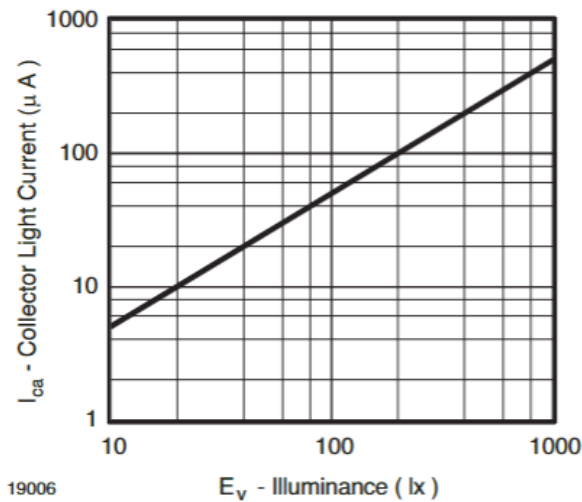


Fig 5. Corriente de colector en función de la luminancia.

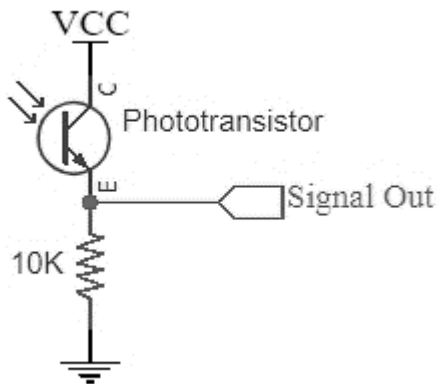


Fig 6. Diagrama esquemático del módulo TEMT6000

c) Sensor de luminosidad BH1750

Para optimizar la medición de luminosidad, se incorporó el módulo BH1750, el cual ofrece múltiples resoluciones programables. Se seleccionó la configuración de mayor precisión (0.5 lux), lo que permite corregir el error y calibrar las lecturas del sensor TEMT6000 cada vez que se disponga de una medición válida del BH1750.

El módulo BH1750 emplea un fotodiodo cuya respuesta espectral se asemeja a la del ojo humano. Este fotodiodo

convierte la luz ambiental en una corriente eléctrica proporcional a la intensidad luminosa (lux), mediante el incremento de portadores minoritarios en el semiconductor. Dicha corriente es procesada por una etapa de integración basada en un amplificador operacional, seguida de un convertidor corriente-tensión.

La señal resultante es digitalizada mediante un convertidor analógico-digital (ADC) de tipo sigma-delta de 16 bits, lo que permite obtener mediciones de alta resolución en un amplio rango de iluminancia (1 a 65.535 lx). Finalmente, los datos son gestionados por una unidad lógica interna y almacenados en un registro accesible a través de una interfaz I²C (ver Fig 8).

- Rango entre 1 a 65535 lux.
- Resolución hasta 1 lux.
- Precisión absoluta típica de $\pm 20\%$ de la lectura.
- Variación en la repetibilidad menor al 1-2%.
- Comunicación I²C.
- Rechazo de frecuencias 50/60 hz.

Measurement Mode	Measurement Time.	Resolution
H-resolution Mode2	Typ. 120ms.	0.5 lx
H-Resolution Mode	Typ. 120ms.	1 lx.
L-Resolution Mode	Typ. 16ms.	4 lx.

Fig 7. Tabla de resolución/tiempo de medición.

- Modo de medición: se selecciona mediante un opcode
 - 0x10 → Modo continuo alta resolución (1 lx).
 - 0x11 → Modo continuo alta resolución 2 (0.5 lx).
 - 0x13 → Modo continuo baja resolución (4 lx).
- Tiempo de medición (MTreg): se puede ajustar para modificar la sensibilidad.
- Comando de encendido (0x01) y apagado (0x00).
- Comando de reset (0x07) para limpiar el registro de datos.

Pinout (ver Fig 8):

- **VCC:** Pin de alimentación del módulo (3V3).
- **GND:** Pin de tierra.
- **SCL:** Pin de clock periférico I²C.
- **SDA:** Pin de transmisión de datos
- **ADDR:** Pin de cambio de dirección I²C

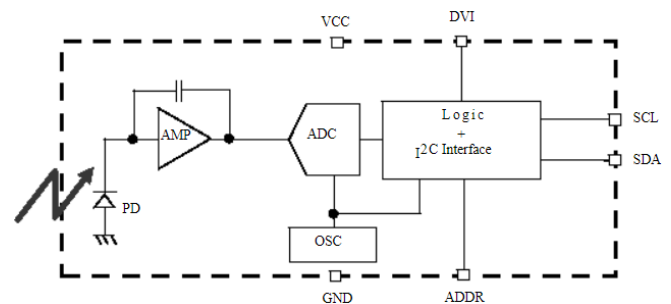


Fig 8. Diagrama en bloque del BH1750



Fig 9. Foto del sensor BH1750

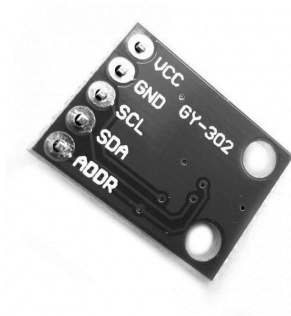


Fig 10. Pinout sensor BH1750

- d) Reloj de tiempo real DS1307
- Comunicación I2C.
 - Incluye memoria EEPROM de 1Kbit AT24C01C.

Pinout:

- **VCC:** Pin de alimentación del módulo (3V3).
- **GND:** Pin de tierra.
- **SCL:** Pin de clock periférico I2C.
- **SDA:** Pin de transmisión/recepción de datos.
- **ADDR:** Pin de cambio de dirección I2C.
- **BAT:** Tension de batería
- **DS:** Proporciona lecturas de temperatura en caso de incluir DS18B20.
- **SQ:** Emite señal cuadrada

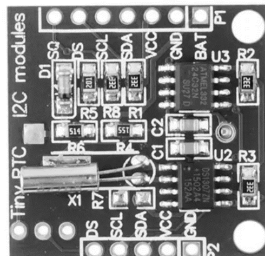
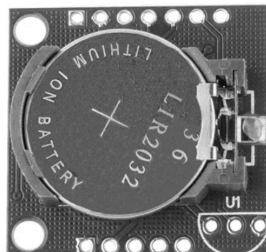


Fig 11. Módulo ds1307.

- e) Display OLED
- Resolución de 128x64

- Comunicación I2C.



Fig 12. Módulo ds1307.

Pinout:

- **VCC:** Pin de alimentación del módulo (3V3).
- **GND:** Pin de tierra.
- **SCL:** Pin de clock periférico I2C.
- **SDA:** Pin de transmisión/recepción de datos.

G. SOFTWARE

G.1 Tareas

El software embebido en la Raspberry Pi Pico 2 cuenta con varias tareas, enfocadas al principio de responsabilidad única, para asegurar la modularidad y portabilidad de las mismas a otros sistemas microcontrolados. El mismo cuenta con una tarea guardiana, la cual resguarda el acceso al recurso del bus I2C, esta tarea guardiana funciona aplicando conceptos del patrón de diseño strategy el cual consiste en un bloque de código que recibe una estrategia a ejecutar, luego en caso de que la estrategia lo requiera, la `i2c_guardian_task` enviará el contexto de la estrategia seleccionada, a su vez cuenta con una tarea guardiana la cual resguarda el acceso al puerto UART el cual es utilizado para su comunicación con la Raspberry Pi 5.

Lista de tareas

Tarea	Responsabilidad
<code>get_lux_task</code>	Obtener el valor de luminosidad en Lux
<code>control_task</code>	Controlar la luminosidad
<code>bh1750_task</code>	Envía la función a ejecutar a la tarea guardiana para obtener los datos del BH1750
<code>rtc_task</code>	Envía la función a ejecutar a la tarea guardiana para obtener los datos del DS1307 o modificar su configuración
<code>storage_task</code>	Almacena los datos junto la informacion del dia y reloj
<code>user_task</code>	Se encarga de manejar el display

btms_task	Se encarga de revisar los botones de la interfaz de usuario
i2c_guardian_task	Utiliza el bus I2C
cli_task	Se encarga de manejar el puerto UART y permitir al usuario acceder a la configuración

El flujo principal del programa está enfocado en tomar los valores de los sensores, realizar filtrado y acondicionamiento de las señales, enviarlos por una cola q_control la cual será recibida por la tarea control_task que a su vez recibirá la configuración del setpoint del usuario en la cola q_set_point, estos valores serán utilizados para calcular el control PID, controlando la luminosidad del LED.

Para la interfaz de usuario a nivel de hardware habrá una tarea btms_task, la cual será la encargada de revisar el estado de los GPIO, calcular el incremento del encoder y notificar a la tarea user_task a través de semáforos binarios y semáforos contadores. La tarea user_task tomará la decisión de que mostrar en la interfaz de usuario y gestionar los recursos del usuario (configuración y vista). El menú del usuario gestionado por user_task contiene 4 modos (ver Anexo A):

- Principal: El usuario elige un set point del tipo escalón o rampa.
- Configuración: El usuario puede modificar la hora del RTC y el valor de mínimo y máximo indicador Led.
- Log: El usuario puede enviar o borrar los logs almacenados en la memoria flash interna.

A su vez el sistema cuenta con una tarea encargada de la gestión del puerto UART, el cual al recibir datos los compara contra un conjunto de comandos los cuales podrá utilizar el usuario para chequear el estado del dispositivo y configurarlo.

Esta comunicación UART será imprescindible para la interconexión entre el sistema operativo de tiempo real con el sistema operativo de propósito general.

G.2 Sistema operativo de Propósito general

Dentro del GPOS se incluyó un módulo de Kernel capaz de utilizar el puerto UART de la Raspberry Pi 5, este módulo permite al usuario interactuar con el hardware a través de un Char Device, al mismo se incluyeron Files Operations las cuales permiten al usuario leer y escribir en el puerto UART a través de la escritura y lectura del mismo. Este puerto es manejado por el Kernel haciendo uso de device trees a través de un overlay personalizado.

G.3 Interfaz de usuario

El usuario será capaz de interactuar con el periférico (EGA) a través de una aplicación web, la misma se comunica con una web API realizada con FastAPI embebida en la Raspberry Pi 5, la cual hace uso de Websockets para la publicación de datos recibidos a través del driver desarrollado (ver Anexo B). Por otro lado el usuario tendrá acceso a una página Web servida en la misma plataforma la

cual permitirá al usuario modificar parámetros de una manera gráfica y amigable a nivel de usuario. (ver Anexo J)

G.4 Control PID

Este sistema está compuesto por un control PID, que consiste en un control conformado por tres componentes: proporcional, integral y derivativo. Lo que nos permitirá mantener un control estable y que nuestra variable a monitorear se acerque al valor deseado. En nuestro proyecto este control está representado por la siguiente fórmula.:

$$pid = error * kp + diferencial_error * kd + integral_error * ki$$

Donde:

$$error = set\ point - entrada$$

$$diferencia_error = error - error_{t-1}$$

$$integral_error = integral_error + error * dt$$

Proporcional:

- Responde al error actual.
- Cuanto más grande es el error, más grande es la corrección.
- Kp es una constante que ajusta la fuerza de la respuesta proporcional, y la velocidad del control.

Integral

- Responde a la acumulación del error en el tiempo.
- Corrige el error en el régimen transitorio.
- Ki siendo la constante del término integral.
- Gran peso en régimen permanente.

Derivativo

- Responde a la tasa de cambio del error
- Ayuda a estabilizar el sistema evitando sobre correcciones.
- Kd es la constante del término derivativo.
- Gran peso en régimen transitorio.

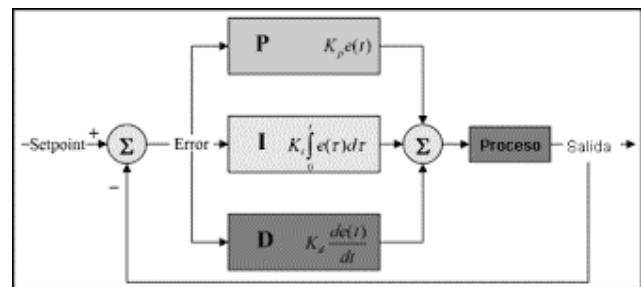


Fig 13. Diagrama en bloque de control PID

H. Conclusiones

El equipo logra controlar los niveles de iluminancia con un error absoluto de ± 10 lux y un error en régimen permanente del 5% y se cumplió el objetivo de la cátedra de implementar un sistema de control basado en un RTOS. Durante el proceso de diseño e implementación se identificaron diversas problemáticas, entre ellas el ruido registrado en el sensor TEMT6000. Para mitigar este efecto se implementó una estrategia de fusión de sensores junto con un filtro de Kalman motivado con fines académicos, esto nos permitió disminuir el error absoluto inicial de ± 135 lux a ± 10

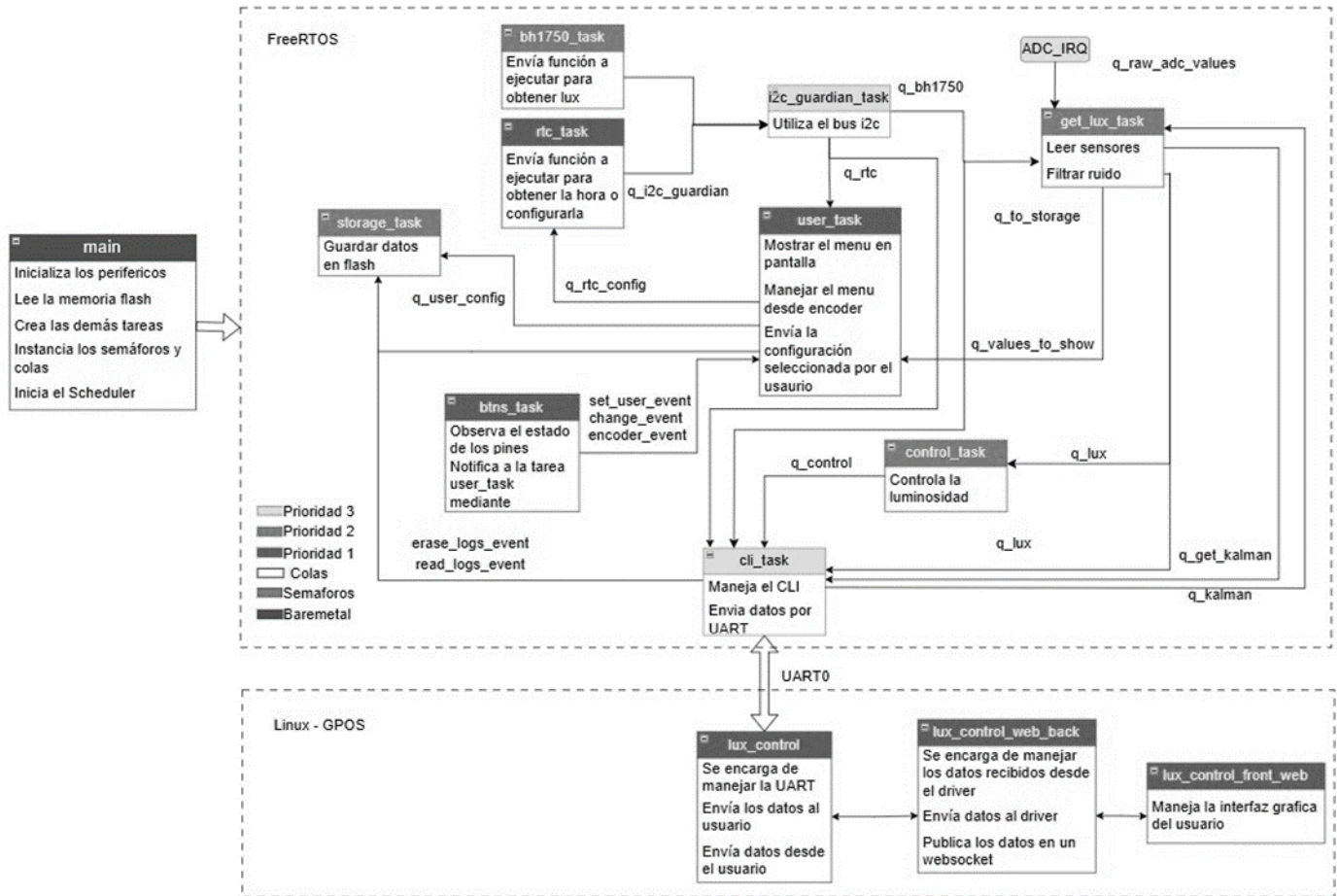
lux. Como línea futura de mejora, se propone modificar el modelo predictivo utilizado en el filtro o reemplazar el tipo de filtro, con el objetivo de optimizar los tiempos de respuesta y aumentar la robustez del sistema ante variaciones rápidas en las condiciones de iluminación.

J. REFERENCIAS

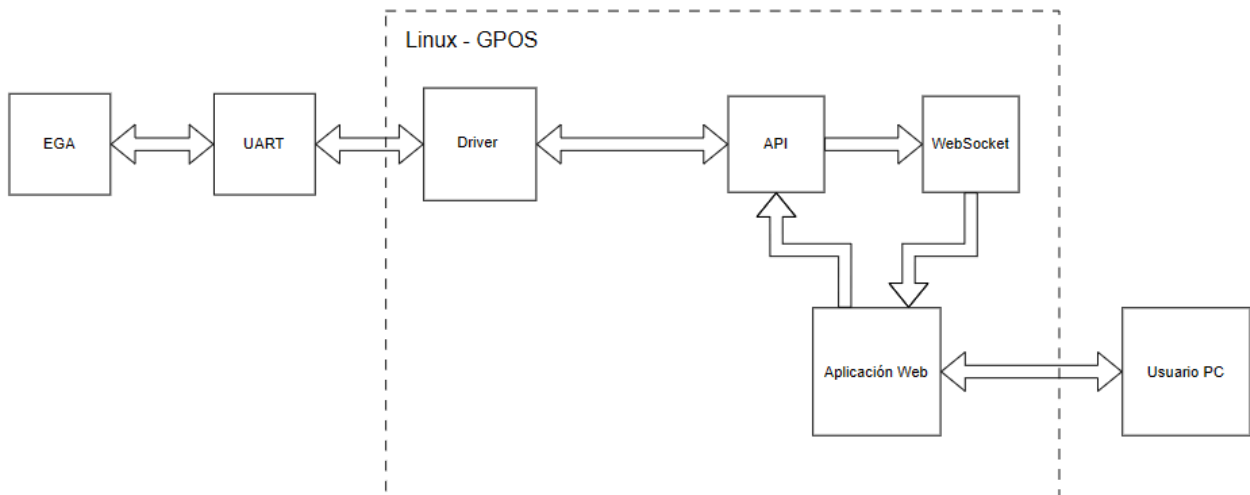
- [1] “Raspberry pi pico 2 W documentation,” Raspberry Pi,
<https://www.raspberrypi.com/>
- [2] “BH1750 data sheet,” Rrohmn Semiconductor,
<https://www.alldatasheet.com/datasheet-pdf/view/338083/ROHM/BH1750FVI.html>
- [3] “TEMT6000 data sheet,” Vishay Siliconix,
<http://www.vishay.com>
- [4] “DS1307 data sheet”
<https://www.alldatasheet.com/datasheet-pdf/pdf/58481/DALLAS/DS1307.html>

Anexos:

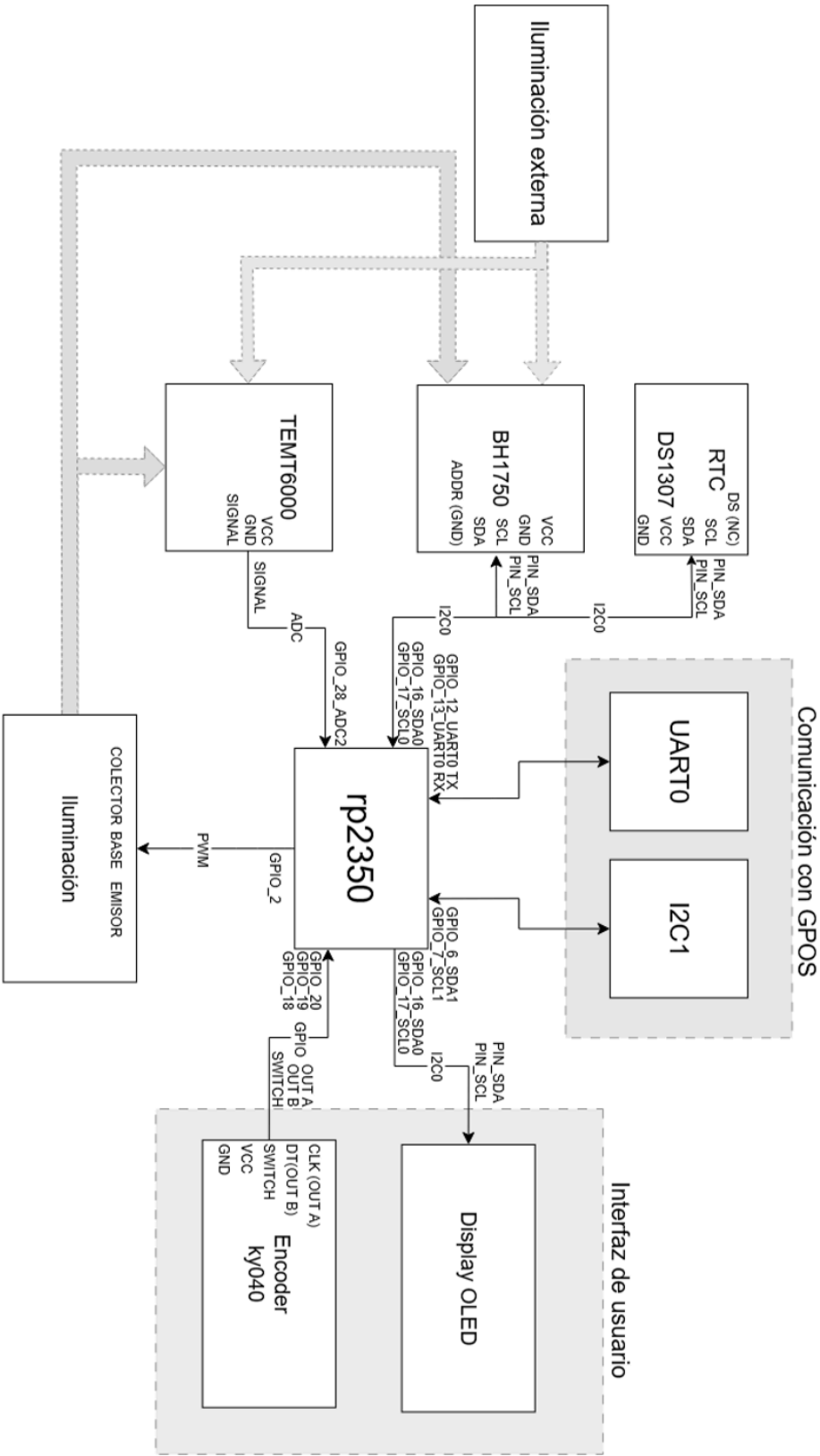
Anexo A: Guía de código



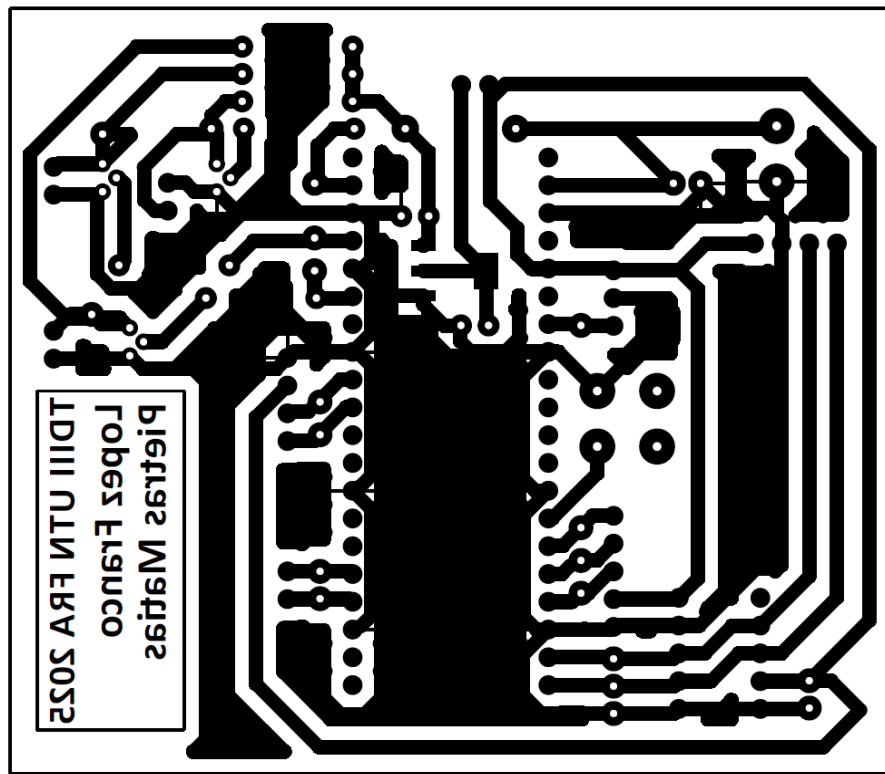
Anexo B: Interacción entre EGA y EGB



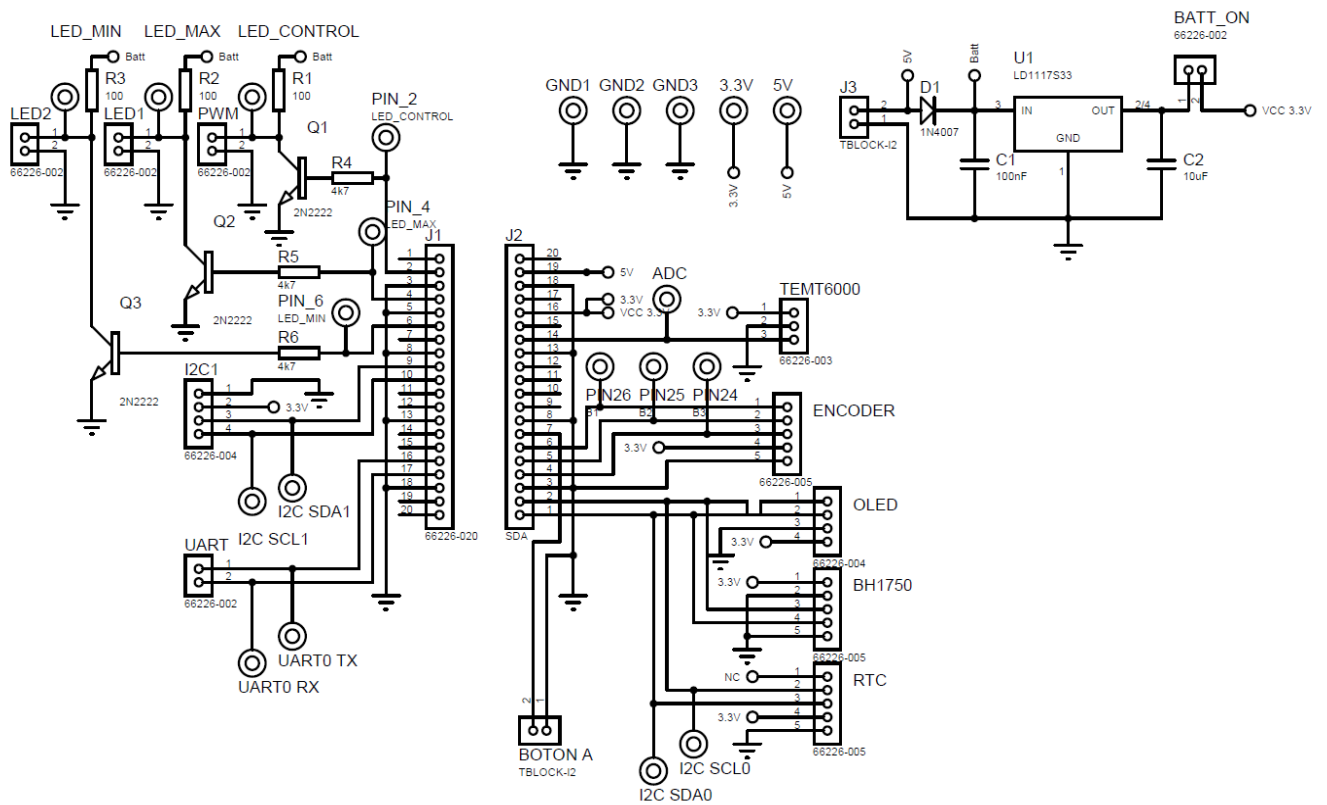
Anexo C: Diagrama en bloques



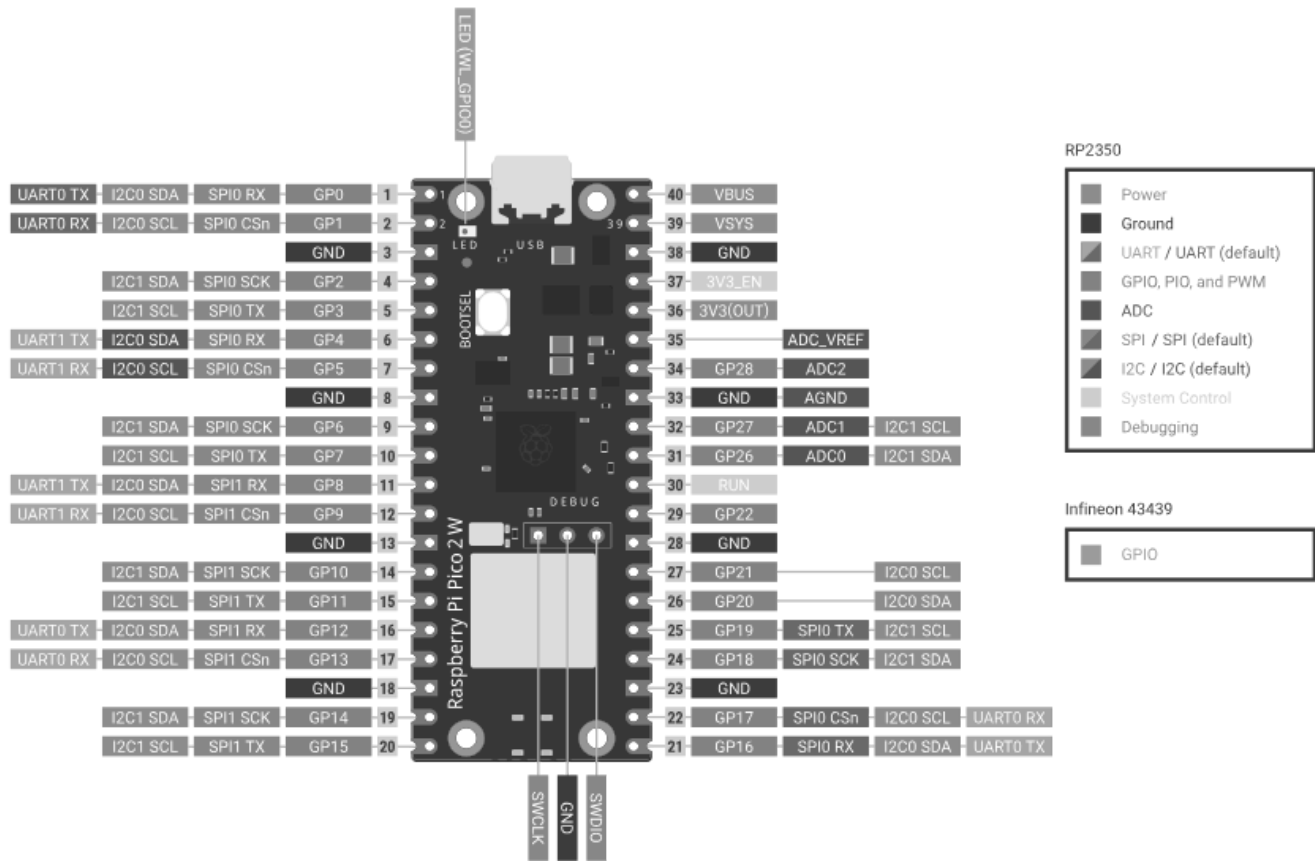
Anexo D: PCB



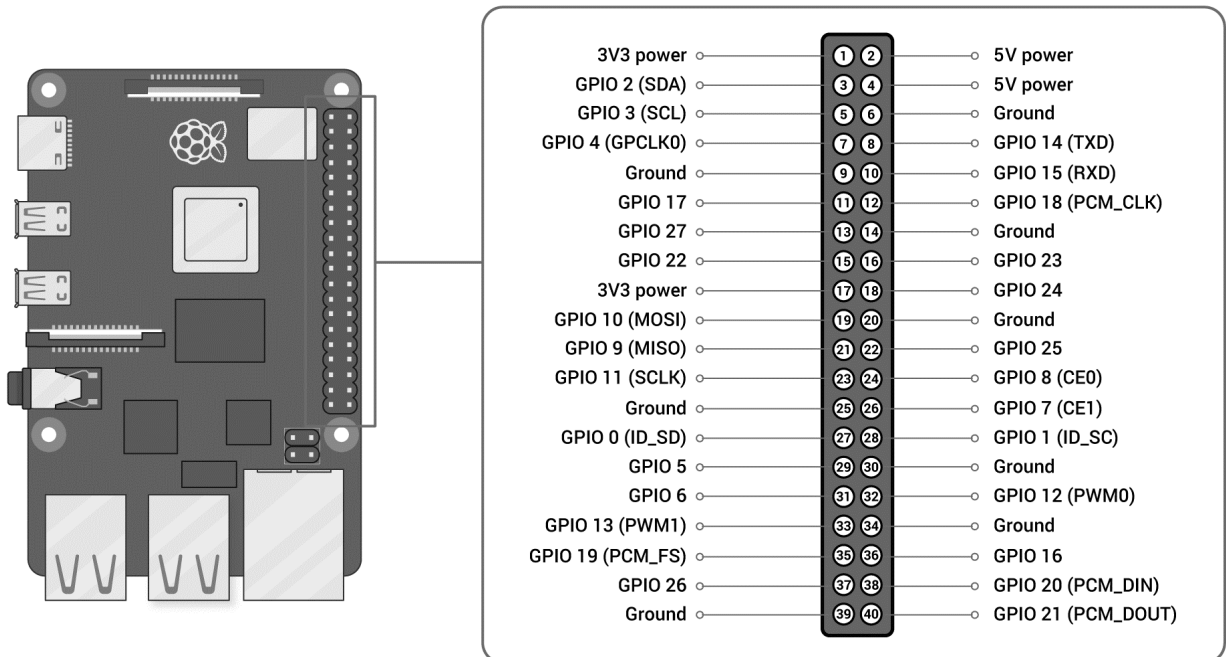
Anexo E: Diagrama esquemático



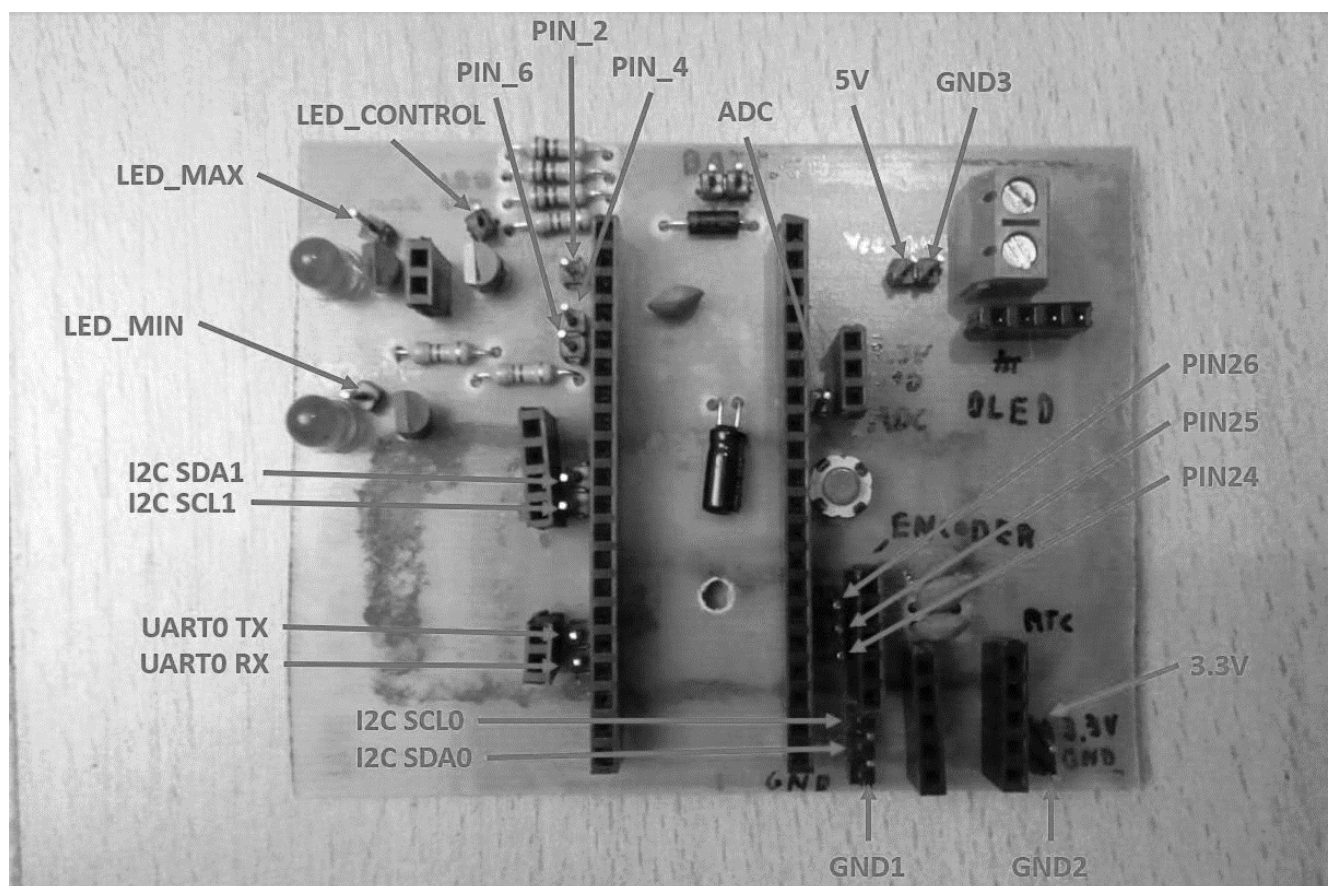
Anexo F: Pinout Raspberry pi pico 2W



Anexo G: Pinout Raspberry pi 5



Anexo H: Test points



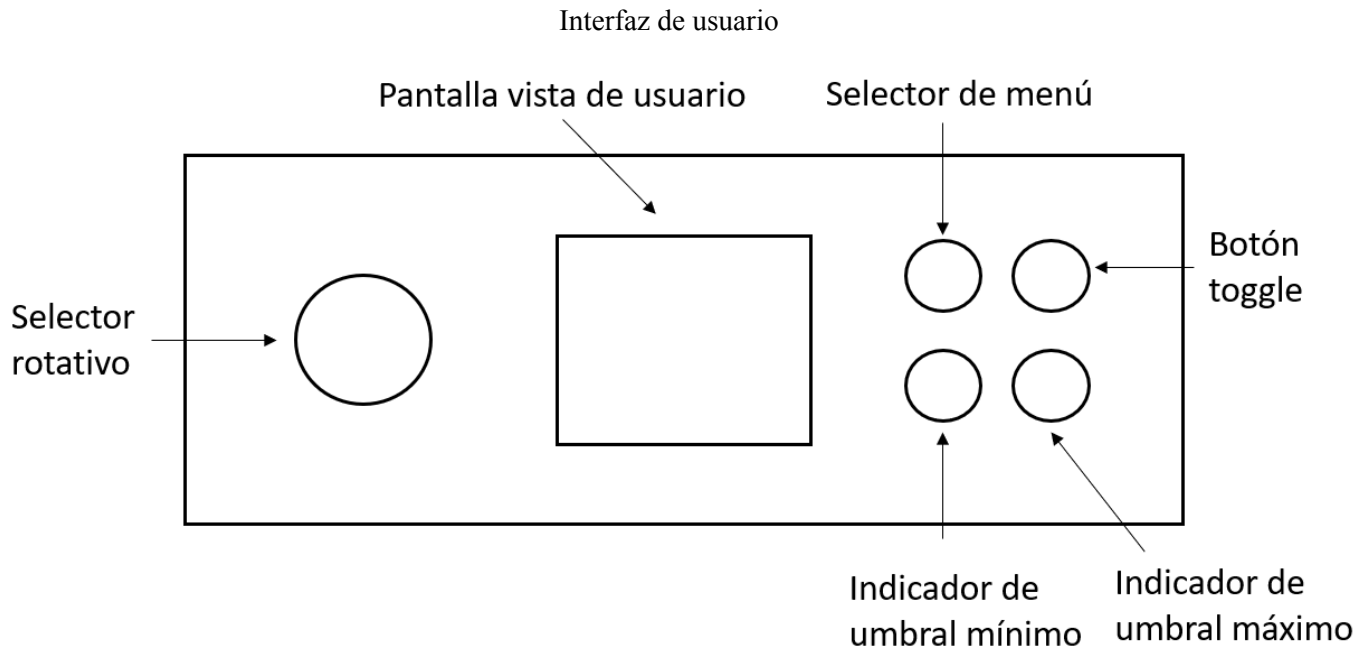
Anexo I: Listado de comandos implementados

Comando	Descripción
toggle control	Activa o desactiva el PWM
logs	Obtiene los logs guardados en la memoria Flash
user_params	Obtiene los parámetros del usuario
pid_params	Obtiene los parámetros del PID
filter_params	Obtiene los parámetros del filtro de Kalman y de media
set_point	Obtiene o envía el setpoint
set_point f	Obtiene o envía el setpoint final
rise time	Obtiene o envía el tiempo de la rampa
pwm	Obtiene el valor de PWM
min	Obtiene o envía el umbral mínimo
max	Obtiene o envía el umbral máximo
bh1750	Obtiene el valor del bh1750
temt6000	Obtiene el valor del temt6000
pid kp	Obtiene o envía el valor kp
pid ki	Obtiene o envía el valor ki
pid kd	Obtiene o envía el valor kd
calib	Obtiene o envía el valor calib
alpha	Obtiene o envía el valor alpha
kalman R	Obtiene o envía el valor kalman R
kalman Q	Obtiene o envía el valor kalman Q
rtc	Obtiene el valor RTC
hour	Envía el valor hour
minute	Envía el valor minute
second	Envía el valor second
day	Envía el valor day
month	Envía el valor month
year	Envía el valor year

Anexo J: Guía de usuario

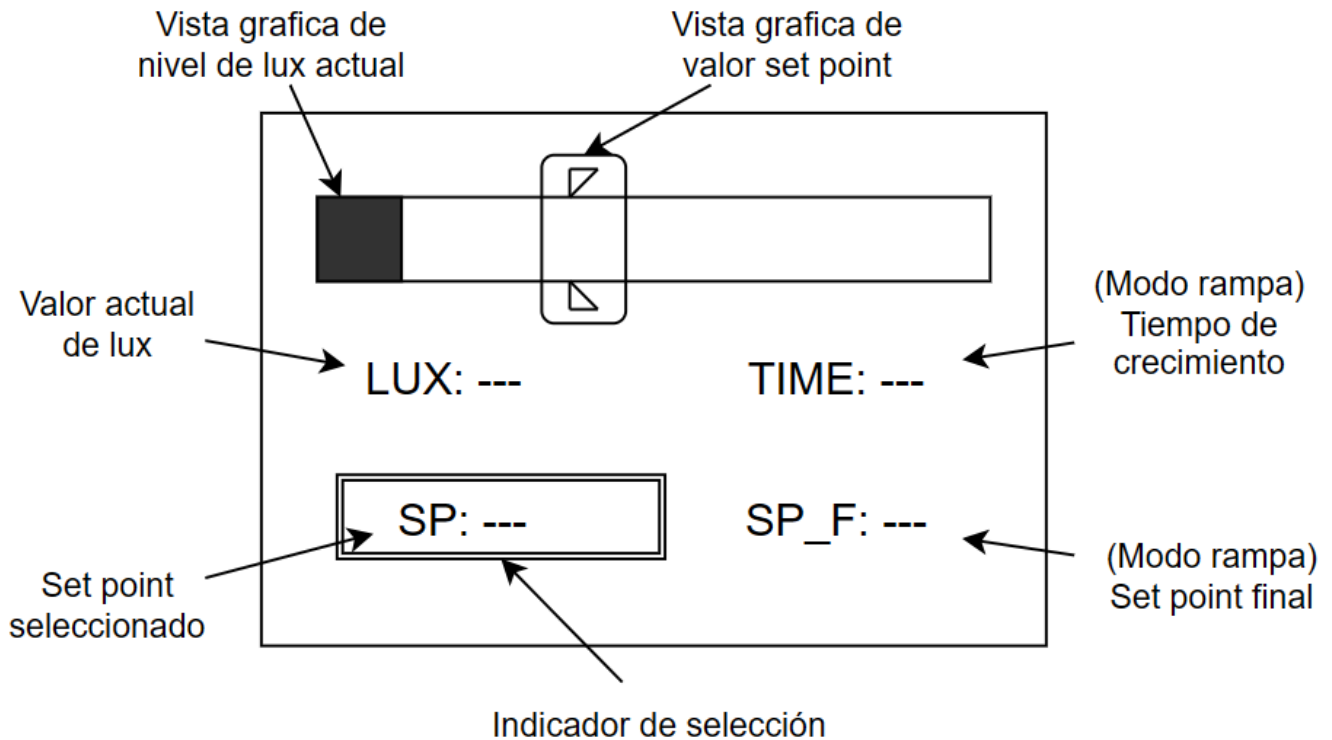
Especificaciones:

- Tensión de alimentación 5V.
- Control de luminosidad de tipo escalón y rampa (100 - 1000 lux).
- Configuración de nivel máximo.
- Configuración de nivel mínimo.
- Almacenamiento de datos cada 5 minutos.
- Captura actual de datos.



- Indicador sobrepaso umbral máximo: Se encenderá en caso de que la medición sobrepase el límite establecido por el usuario.
- Indicador por debajo de umbral mínimo: Se encenderá en caso de que la medición se encuentre por debajo del límite establecido por el usuario.
- Selector rotativo: Selector el cual puede ser presionado.

Vista menú principal



Vista gráfica de nivel de lux actual: Indica gráficamente el nivel de lux actual.

Vista gráfica de valor de set point: Indica gráficamente el set point seleccionado por el usuario.

Valor actual de lux: Indica el valor medido por el sensor en lux.

Set point seleccionado: Set point seleccionado por el usuario.

Tiempo de crecimiento: Valor de tiempo de crecimiento establecido por el usuario.

Set point final: Valor de set point final ingresado por el usuario.

Indicador de selección: Muestra el valor que se está seleccionando.

Modo de uso: Escalón

1. Rotar selector rotativo hasta que la indicación esté posicionada sobre el SP (Set point seleccionado).
2. Presionar el selector rotativo (una vez presionado verá como el recuadro cambia de color).
3. Rotar selector rotativo hasta el valor deseado de set point (A medida que varíe su valor verá como la vista gráfica de valor de set point variará su posición).
4. Una vez posicionado en el valor deseado de set point presionar nuevamente el selector, esto comenzará la acción de control.

***Nota:** Verá cómo varía la intensidad medida a través de la vista gráfica de nivel de lux actual, la cual funcionará a modo de corroboración de seguimiento del set point*

Modo de uso: Rampa

1. Rotar selector hasta que la indicación esté posicionada sobre el SP (Set point seleccionado).
2. Presionar el selector rotativo, una vez presionado verá como el recuadro cambia de color.
3. Rotar selector rotativo hasta el valor deseado de set point (A medida que varíe su valor verá como la vista gráfica de valor de set point variará su posición).
4. Presionar el selector para confirmar el set point inicial.

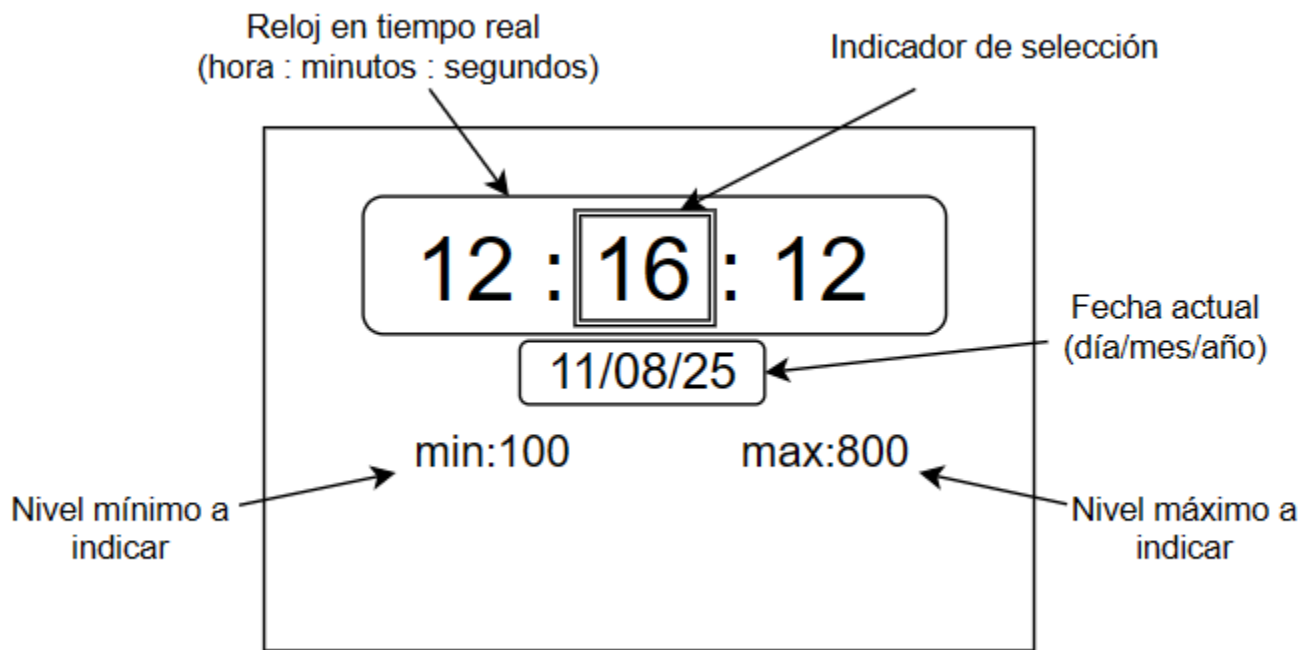
5. Rotar el selector rotativo hasta que el indicador de selección esté posicionado en SP_F(set point final)
6. Rotar selector hasta el valor deseado de set point final.
7. Presionar el selector rotativo para confirmar el set point final.
8. Presionar el selector rotativo para confirmar el tiempo de crecimiento, esto comenzará la acción de control.
9. Rotar el selector rotativo hasta que el indicador de selección esté posicionado en TIME (tiempo de crecimiento de rampa).
10. Rotar selector rotativo hasta el valor deseado de tiempo de crecimiento de rampa.
11. Presionar el selector rotativo para confirmar el tiempo de crecimiento, esto comenzará la acción de control.

***Nota:** Verá cómo varía la intensidad medida a través de la vista gráfica de nivel de lux actual, la cual funcionará a modo de corroboración de seguimiento del set point*

Modo de uso: Captura actual

1. Rotar selector rotativo hasta que la indicación esté posicionada sobre “LUX” (Valor actual en lux).
2. Presionar el selector rotativo (una vez presionado verá como el recuadro cambia de color).

Vista menú de configuración usuario



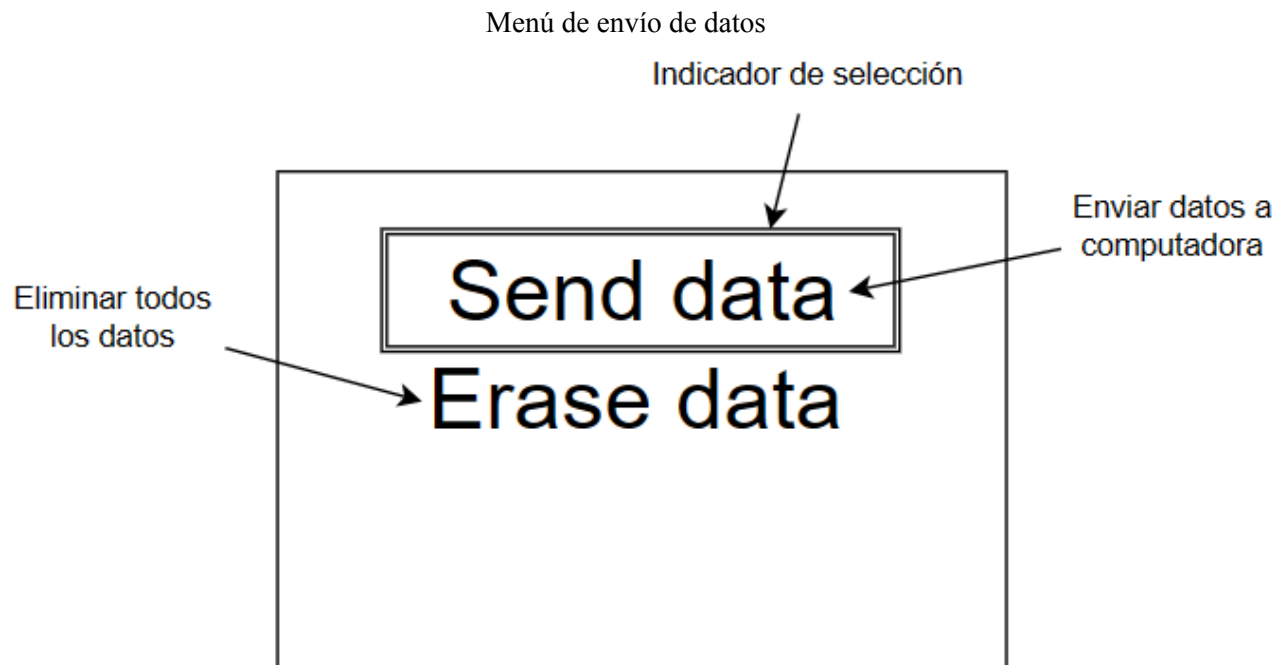
Configuración: Reloj en tiempo real y fecha

1. Rotar selector rotativo hasta que la indicación esté posicionada sobre la magnitud a modificar.
2. Presionar el selector rotativo, una vez presionado verá como el recuadro cambia de color.
3. Rotar selector rotativo hasta el valor deseado.
4. Presionar el selector rotativo para confirmar el cambio.

Configuración: Nivel mínimo a indicar y Nivel máximo a indicar

1. Rotar selector rotativo hasta que la indicación esté posicionada sobre la magnitud a modificar.
2. Presionar el selector rotativo, una vez presionado verá como el recuadro cambia de color.

3. Rotar selector rotativo hasta el valor deseado.
4. Presionar el selector rotativo para confirmar el cambio.



Enviar datos:

1. Asegúrese de que el dispositivo esté conectado a una computadora a través del puerto de conexión a computadora (Ver sección interfaz de usuario).
2. Desde su computadora conectarse al puerto serial COM desde su terminal preferido (ej PuTTY).
3. Rotar el selector rotativo hasta que el indicador de selección esté posicionado sobre “Send data”.
4. Presionar el selector rotativo, una vez presionado verá como el recuadro cambia de color.
5. Presionar el selector rotativo para confirmar el envío de datos, verá en consola los datos almacenados por el equipo.

Eliminar todos los datos:

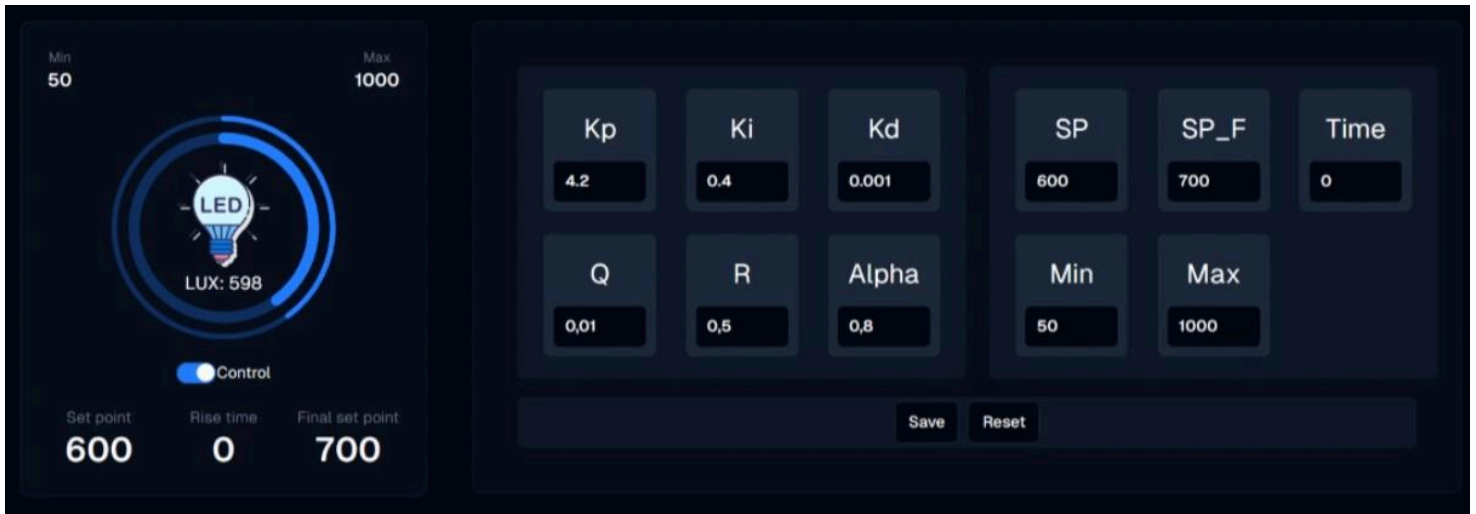
Atención esto eliminará todos los datos del equipo sin posibilidad de recuperarse

1. Rotar el selector rotativo hasta que el indicador de selección esté posicionado sobre “Erase data”.
2. Presionar el selector rotativo, una vez presionado verá como el recuadro cambia de color.
3. Presionar el selector rotativo para confirmar la eliminación de los datos.

Interfaz web

Con el objetivo de llevar un control remoto, se llevó a cabo la siguiente aplicación web realizada con Typescript utilizando el framework de Nextjs, la cual se comunica a un a API realizada con FastAPI que permite el acceso a un WebSocket.

Donde el usuario puede leer y/o modificar los siguientes parámetros:



SP: Set Point inicial o valor deseado

SP_F: Set Point Final, se utiliza para dar el valor final en caso de querer ingresar son una rampa

Time: Tiempo entre el SP y el SP_F en el caso de una rampa.

Min: Umbral mínimo

Max: Umbral máximo

Kp: Constante proporcional del control PID.

Ki: Constante integral del control PID.

Kd: Constante derivativa del control PID.

Q: Constante del filtro de Kalman.

R: Constante del filtro de Kalman.

Alpha: Constante del filtro de Kalman.